CampusGrid: A Hierarchical Spatial Model, Dual-Vector Conflict Detection and Bipartite Anti-Cheating Seating for Campus Space, Timetable and Examination Management

Nikhil Kumawat, Palak Nandani, Kashish Upadhyay, Priyanshu Kumar and Bhawni Kumari
Department of Artificial Intelligence & Data Science, Arya College of Engineering & I.T.
Project guide: Er. Ram Babu Buri, Assistant Professor, Department of Artificial Intelligence & Data Science

***Abstract*—Higher-education institutions often record rooms as flat text entries, build timetables in spreadsheets and seat examination candidates with hand-made door charts. This leaves three operational gaps: software cannot tell usable desks from aisles or damaged terminals; clashes across rooms, instructors and student sections surface only after classes begin; and examination halls are filled without separating candidates who sit the same paper. This paper presents CampusGrid, a web-based Java 17 / Spring Boot 3 system backed by MySQL that addresses all three. CampusGrid models campuses, buildings, floors, rooms and a per-room seat matrix whose cells follow a four-state automaton, so that only available seats count toward capacity. A dual-vector conflict engine validates each candidate class session against room, instructor and student-section mutual-exclusion constraints using a half-open interval overlap test and returns an explainable diagnosis. A deterministic checkerboard interleaving algorithm assigns candidates to seats so that no two candidates of the same paper occupy orthogonally adjacent seats, supported by a proof based on 2-colouring of the grid graph. Access is protected by stateless JWT authentication, BCrypt hashing and role-based access control. In the reported experiments, conflict checks complete in under 3 ms for schedules of up to 500 classes, seating allocation scales linearly at roughly 1,300 seats per second with no adjacency violations, and a 140-test suite passes. Limitations, notably diagonal adjacency and per-cohort feasibility, are discussed.**

***Index Terms*—timetabling, examination seating, conflict detection, Spring Boot, JWT, role-based access control, campus management, bipartite graph.**

I. INTRODUCTION

Universities run on three physical resources: rooms, people and time. In many institutions these are still managed in disconnected tools. Rooms live in flat records such as ‘Room 101, capacity 60’; timetables are assembled in spreadsheets; and examination seating is posted on door charts prepared by hand. Each tool is simple, but together they leave three recurring problems.

The spatial disconnect. A flat capacity figure cannot distinguish a usable desk from a pedestrian aisle, a podium void or a damaged terminal. Capacity is over-estimated, and the error becomes visible only when an examination hall is actually filled.

The dual-vector collision problem. A class session needs a room, an instructor and a student section at the same moment. Clashes occur along a spatial vector (two sections in one room) and a human vector (an instructor or a section in two places at once). Manual scheduling finds many of them only after classes have started.

Examination seating and dishonesty risk. Seating hundreds of candidates without systematic separation places candidates of the same paper side by side, which enables visual copying and paper swapping, while ad-hoc charts cause congestion at the hall door.

The timetabling literature is extensive [1]–[3], but it concentrates on generating or optimising timetables, usually at the granularity of events, periods and rooms. Practical campus software must also (i) model space at seat level, (ii) give immediate, explainable feedback when an administrator edits a schedule, and (iii) produce seating that is provably separated, all inside an authenticated web application that a small institution can deploy cheaply. CampusGrid was built to meet those requirements. The contributions of this paper are:

1) A spatial model S = (C, B, F, R, M) of campuses, buildings, floors, rooms and a per-room seat matrix, in which every seat follows a four-state automaton and only AVAILABLE seats contribute to effective capacity.
2) A dual-vector conflict-detection engine, specified by a half-open interval overlap lemma and three hard mutual-exclusion constraints, that returns a diagnosis vector rather than a bare boolean.
3) A deterministic bipartite checkerboard interleaving algorithm for examination seating, with a correctness theorem and an explicit feasibility condition.
4) A secured, containerised implementation using stateless JWT authentication, BCrypt hashing and role-based access control.
5) An empirical evaluation (latency, throughput and a 140-test verification suite) together with a candid account of limitations.

The rest of the paper is organised as follows. Section II reviews related work, Section III describes the architecture and spatial model, Section IV presents the algorithms and proofs, Section V covers implementation and security, Section VI reports the evaluation, Section VII discusses implications and limits, and Section VIII concludes.

II. RELATED WORK

A. Timetabling surveys and solution paradigms

Timetabling has long been treated as a constraint-satisfaction and combinatorial optimisation problem. Schaerf [1] surveyed its formulations and the techniques applied, including genetic algorithms, tabu search and constraint satisfaction, and Lewis

[2] reviewed metaheuristic approaches, classifying how they treat constraints of differing importance. A recent survey of university course timetabling [3] finds that metaheuristics are the most popular family, followed by hybrid and hyper-heuristic methods, and notes that state-of-the-art methods are not fully used in real-world settings, possibly because of limited flexibility. A representative hybrid is the genetic algorithm with tabu search of Jat and Yang [4] for post-enrolment course timetabling, evaluated on international competition benchmarks.

B. Examination timetabling and anti-cheating

Examination timetabling has its own surveys [5]–[7]. Al-Yakoob et al. [8] give mixed-integer models for exam timetabling and proctor assignment at Kuwait University under gender-based policies. Köksalmış et al. [9] embed cheating- and fatigue-prevention constraints in an integer-programming timetable for an air force academy and report that maximising room utilisation increased it by 55% and reduced the classrooms required by 43%. Based on the abstracts and system descriptions we reviewed, these works operate on exams, periods and rooms; none represents individual seat coordinates.

C. Space utilisation

Beyrouthy et al. [10] study teaching-space utilisation, defined as the share of available seat-hours that are used, and show that real utilisation is often only 20–40% and that the feasibility of allocating a set of courses shows threshold behaviour. Their work motivates accurate, seat-aware capacity figures such as the effective capacity introduced here.

D. Java-based timetabling systems

The closest system-level comparator is UniTime [11], an open-source, web-based system built on J2EE and used at Purdue University. It covers course timetabling, examination timetabling and student sectioning with a constraint-based solver. Timefold [12], the successor of OptaPlanner, is a Java constraint-solver framework widely used for scheduling problems. Both solve the generation and optimisation problem. CampusGrid has a different scope: it validates administrator-entered sessions deterministically and allocates examination seats constructively, and it could sit in front of, or alongside, such a solver.

E. Access control

CampusGrid's security layer builds on role-based access control [13], [14], JSON Web Tokens [15], [16] and adaptive password hashing [17].

TABLE I
POSITIONING OF CAMPUSGRID AGAINST REPRESENTATIVE WORK

Work	Main task	Approach	Platform	Seat-level model
Jat and Yang [4]	Course timetabling	GA + tabu search	–	Not reported
Al-Yakoob et al. [8]	Exam timetabling, proctors	Mixed-integer programming	–	Not reported
Köksalmış et al. [9]	Exam timetabling with anti-cheating	Integer programming	–	Not reported
UniTime [11]	Course and exam timetabling	Constraint solver, local search	Java / J2EE	Not reported
Timefold [12]	Generic scheduling	Constraint solver	Java	N/A
CampusGrid	Space, validation, exam seating	Deterministic checks + checkerboard	Java 17, Spring Boot 3, MySQL	**Yes (4-state matrix)**

'–': not stated in the sources reviewed. 'Not reported': not described in the abstracts or system descriptions reviewed.

III. SYSTEM ARCHITECTURE AND SPATIAL MODEL

A. Layered architecture

CampusGrid follows a conventional layered design (Fig. 1). A browser client written in HTML5, CSS3 and JavaScript calls Spring MVC controllers. A security filter (JwtAuthFilter) authenticates each request before it reaches the service layer. Domain services (SpaceManagementService, TimetableCollisionEngine, SeatingLayoutService and JwtService) hold the business rules. Spring Data JPA repositories persist to MySQL 8.0 (InnoDB) through a HikariCP pool with a maximum of 20 connections, a 30 s connection timeout and a 10 min idle timeout.

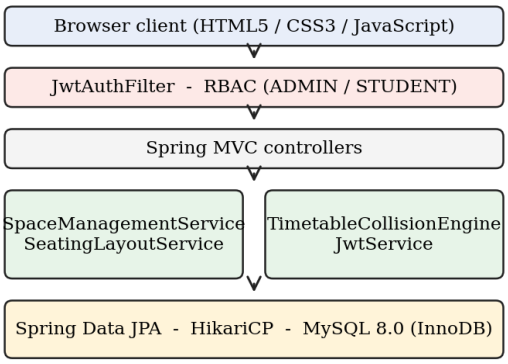


Fig. 1. Layered architecture of CampusGrid.

The work is divided into five modules: identity management, JWT security and executive telemetry (Module 1); the campus spatial hierarchy and visual seating designer (Module 2); the academic timetable and collision engine (Module 3); examination management and the anti-cheating seating engine (Module 4); and the student portal, faculty hub and UI integration (Module 5).

B. Spatial hierarchy

The physical campus is modelled as the tuple

$$S = (C, B, F, R, M)$$

where C is the set of campuses, B the set of buildings with parent(b) $\in$ C, F the set of floors with parent(f) $\in$ B, R the set of rooms with parent(r) $\in$ F, and M_r the seating matrix of room r. A room carries a code, a type $\tau_r \in$ {CLASSROOM, LAB, AUDITORIUM}, a nominal capacity K_r, a workstation count W_r and grid dimensions $N_{rows} \times N_{cols}$. The structural hierarchy is therefore four-tier (C, B, F, R) with the seat matrix M_r attached below each room.

C. Seat automaton and effective capacity

Each seat s_{ij} in M_r is in exactly one of four states, Σ_s = {AVAILABLE, OCCUPIED, BLOCKED, MAINTENANCE}. Administrators use the visual seating designer to mark aisle gaps, podium voids and damaged terminals as BLOCKED or MAINTENANCE; allocation moves an AVAILABLE seat to OCCUPIED. The effective allocatable capacity of a room is

$$K_{eff}(r) = \Sigma_{i,j} [\ state(s_{ij}) = AVAILABLE\ ] \quad (1)$$

where $[\cdot]$ is 1 when its condition holds and 0 otherwise. Before any allocation no seat is OCCUPIED, so $K_{eff}(r)$ equals the number of usable seats. By construction, aisles and damaged workstations are never counted and never receive a candidate, which removes the over-estimation caused by flat capacity figures.

D. Administrator interface

Fig. 2 shows the deployed administrator interface. Panel (a) is the campus explorer, which lists blocks and floors under a campus together with the room count per floor; the instance shown holds one campus, five blocks and 25 academic spaces. Panel (b) is the seating layout designer: a layout template (here a walkway pattern) and the grid dimensions produce a live preview of the seats that remain once aisle gaps are removed, and the administrator commits the result with a single action. For room A103 (5 × 8 grid, declared capacity 40) the preview calculates 28 seats, which is exactly the over-estimate that a flat capacity figure hides and that Eq. (1) removes.

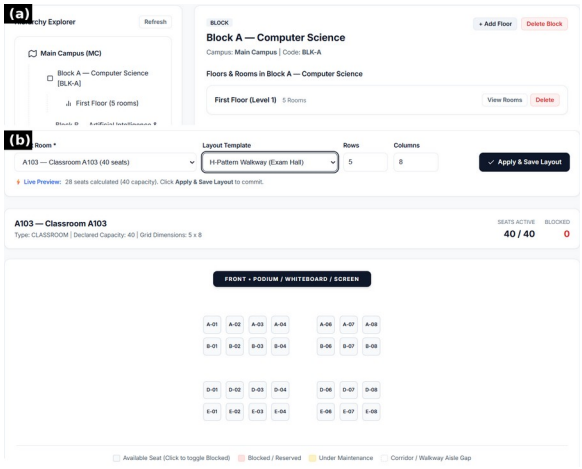


Fig. 2. Administrator interface: (a) campus explorer; (b) seating layout designer for room A103.

IV. ALGORITHMIC FORMULATIONS

A. Timetable model and the interval lemma

A class session is t = (U, G, I, R, D, S, E): course U, student section (cohort) G, instructor I, room R, day D, start S and end E, with S < E. Sessions occupy half-open intervals [S, E), so a class ending at 10:00 and another starting at 10:00 abut without overlapping.

Lemma 1 (overlap). For sessions a and b on the same day,

$$\text{Overlap}(a, b) \Leftrightarrow (S_a < E_b) \wedge (E_a > S_b) \quad (2)$$

Proof. Two half-open intervals are disjoint exactly when one ends no later than the other starts, that is $E_a \le S_b$ or $E_b \le S_a$. Negating gives $S_a < E_b$ and $E_a > S_b$. Abutting intervals with $E_a = S_b$ satisfy the first disjointness condition and therefore do not overlap. ∎

A schedule is valid when no two sessions on the same day violate any of the following hard constraints:

C1 **Room mutex:** $R_a = R_b$ implies ¬Overlap(a, b).
C2 **Instructor mutex:** $I_a = I_b$ implies ¬Overlap(a, b).
C3 **Section mutex:** $G_a = G_b$ implies ¬Overlap(a, b).
C4 **Capacity:** $|G| \le K_{eff}(R)$ for the assigned room.

B. Algorithm 1: dual-vector conflict detection

Given the existing schedule T and a candidate t′ with S′ < E′, Algorithm 1 returns whether t′ is admissible together with a diagnosis vector that names every violated constraint and the session it collides with.

Algorithm 1 ValidateSession(T, t′)

```
Input : schedule T, candidate t' = (U,G,I,R,D,S,E)
Output: (valid, Δ)
1  Δ ← {}
2  if |G| > K_eff(R): add (CAPACITY) to Δ
3  for each active t in T with t.D = D:
4    if S < t.E and E > t.S:        // Lemma 1
5      if t.R = R: add (ROOM, t) to Δ
6      if t.I = I: add (INSTRUCTOR, t) to Δ
7      if t.G = G: add (SECTION, t) to Δ
8  return (Δ = {}, Δ)
```

The spatial vector is checked on line 5 and the human vector on lines 6 and 7, so a single pass reports all clash types at once, which lets the interface tell the administrator exactly what to change.

Complexity. A candidate is compared with at most M_D sessions on its day, so one validation costs O(M) in the worst case. With composite B-tree indexes on (day_of_week, room_id) and (day_of_week, section_id), the candidates for the room and section vectors are retrieved in O(log M + k), where k is the number of same-day rows sharing the key. The instructor vector needs an equivalent (day_of_week, instructor_id) index to behave the same way. Inserting N sessions one by one is $O(N^2)$ in the worst case and close to O(N log N) when k is small.

C. Anti-cheating seating: distance metrics and the isolation invariant

For seats a = (r_a, c_a) and b = (r_b, c_b), the Manhattan and Chebyshev distances are

$$D_M(a, b) = |r_a - r_b| + |c_a - c_b|$$
$$D_\infty(a, b) = \max(|r_a - r_b|, |c_a - c_b|) \quad (3)$$

The adjacency isolation invariant required by the seating engine is

$$\forall\ a \ne b: \text{Exam}(a) = \text{Exam}(b) \Rightarrow D_M(a, b) > 1 \quad (4)$$

Invariant (4) forbids two candidates of the same paper from sitting side by side or front to back. It does not forbid diagonal neighbours, for which $D_M = 2$ and $D_\infty = 1$; this is discussed in Section VII.

Theorem 1 (checkerboard isolation). Partition the seat grid into parity classes V_0 = {(r, c) : (r + c) mod 2 = 0} and V_1 = {(r,

c) : (r + c) mod 2 = 1}. If all candidates of an exam are assigned to seats of a single class, invariant (4) holds for that exam.

Proof. The grid graph, in which each seat is joined to its horizontal and vertical neighbours, is bipartite with parts V_0 and V_1, because adjacent seats differ by exactly one in r + c [18]. Take two distinct seats a and b in the same class. Then $(r_a + c_a)$ and $(r_b + c_b)$ have equal parity, so $(r_a - r_b) + (c_a - c_b)$ is even. Since |x| + |y| and x + y have the same parity, $D_M(a, b)$ is even. Distinct seats have $D_M \geq 1$, hence $D_M \geq 2 > 1$. Removing BLOCKED or MAINTENANCE seats leaves an induced subgraph of a bipartite graph, so the argument is unaffected. ∎

Feasibility condition. Theorem 1 only helps if each exam fits in one class. Let n_e be the number of candidates of exam e, and let a_0 and a_1 be the numbers of AVAILABLE seats in V_0 and V_1 across the allocated rooms. Interleaving is feasible only if the exams can be split into two groups A and B with the sum of n_e over A at most a_0 and the sum over B at most a_1. In particular, a single exam larger than $\max(a_0, a_1)$ can never be isolated by this scheme.

D. Algorithm 2: multi-cohort checkerboard interleaving

Algorithm 2 CheckerboardInterleave(R, Q)

```
Input : rooms R, exam cohorts Q = {Q1, ..., Qu}
Output: Ω: student → (room, r, c, seat) or INFEASIBLE
1  if Σ|Qi| > Σ K_eff(k) over k in R: return INFEASIBLE
2  sort Q by |Qi| descending      // first-fit decreasing
3  put each Qi in group A or B with more spare capacity
   (a0 for A, a1 for B)
4  if some Qi fits neither group: return INFEASIBLE
5  for each room k in R, each cell (r,c) in row-major order:
6    if state(r,c) ≠ AVAILABLE: continue   // aisle, broken
7    P ← (r + c) mod 2
8    if P = 0 and queue(A) non-empty: seat next of A
9    if P = 1 and queue(B) non-empty: seat next of B
10   otherwise leave the seat empty (never cross class)
11 return Ω
```

Line 10 is what preserves Theorem 1: a candidate is never placed in the opposite class, even if that leaves a seat empty. Candidates of different exams within the same group may sit anywhere in their class. When several rooms are used, each room is processed independently, so the candidates of one exam may be split across rooms without breaking the invariant.

Complexity. Algorithm 2 visits each seat once and sorts u cohorts, so it runs in $O(S + u \log u) = O(S)$ for S seats and a small number of cohorts.

A stronger variant (not part of the evaluated implementation). Colouring cell (r, c) by the pair (r mod 2, c mod 2) gives four classes. Two distinct seats of the same class differ by an even amount in both coordinates, so $D_\infty \geq 2$, which also excludes diagonal neighbours. The price is that each exam may use only one quarter of the seats, so this variant suits halls with spare capacity.

V. IMPLEMENTATION AND SECURITY

A. Technology stack

CampusGrid is written in Java 17 LTS (Eclipse Temurin) on Spring Boot 3.3.4 with Spring MVC and Spring Data JPA. Data is stored in MySQL 8.0 Community Server using the transactional InnoDB engine, accessed through a HikariCP connection pool. The front end is a browser client written in HTML5, CSS3 and JavaScript (ES6+) that calls the Spring MVC controllers.

B. Authentication and authorisation

Users authenticate with stateless JSON Web Tokens [15] signed with HMAC-SHA512 (HS512). Access tokens live for 15 minutes (900,000 ms) and refresh tokens for 7 days (604,800,000 ms). RFC 7518 [16] requires an HS512 signing key of at least 512 bits, so deployments must inject a key of that length from the environment rather than from source code. Passwords are stored as BCrypt hashes with work factor 10 [17].

Authorisation follows role-based access control [13], [14] with two roles, ROLE_ADMIN and ROLE_STUDENT. JwtAuthFilter verifies the signature and expiry of each token, extracts the role claim and maps it to the authorities used by route-level rules, so that space design, timetable editing and examination seating are reachable only by administrators while students see their own portal views. Transport protection is delegated to HTTPS termination at the hosting platform, which should negotiate TLS 1.3 [19]; tokens must never travel over plain HTTP.

Two limits follow from the stateless design. A token cannot be revoked individually before it expires, so the short access-token lifetime is the main bound on exposure; refresh-token rotation or a deny-list would tighten this. The two-role model also has no separate faculty role; a ROLE_FACULTY is planned (Section VIII).

C. Containerised deployment

The application is built with a multi-stage Dockerfile. Stage 1 uses the maven:3.9.6-eclipse-temurin-17 image to compile and package the application; stage 2 copies the artefact onto eclipse-temurin:17-jre-alpine, yielding a production image of about 180 MB. The service is deployed on the Railway platform. Configuration is injected through the environment variables PORT, MYSQLHOST, MYSQLPORT, MYSQLDATABASE, MYSQLUSER and MYSQLPASSWORD, so the same image runs locally and in the cloud without code changes.

VI. EXPERIMENTAL EVALUATION

A. Setup

Three experiments were run: (1) the latency of Algorithm 1 as the schedule grows, (2) the throughput of Algorithm 2 as the facility grows, and (3) a suite of 140 automated integration tests. Timetable workloads were synthetic: classes were spread over a five-day week at 10 to 100 classes per day, and clashes were injected by creating overlapping start and end intervals for duplicate room, instructor and student-section assignments (2.0% to 3.6% of sessions in Table II). Timings were taken on a developer workstation running Java 17 [authors: CPU, RAM, database location and number of repetitions, if recorded]. The allocation routine of Algorithm 2 is a pure in-memory computation over the student list and the seat map; persistence to the database is performed separately by the seating layout service.

B. Timetable collision engine

TABLE II
ALGORITHM 1 LATENCY VERSUS SCHEDULE SIZE

N	Pair checks	Mean (µs)	p95 (µs)	Clashes found (injected)	False pos.
50	450	184	240	0	0.00%
100	1,900	312	410	2	0.00%
200	7,800	684	890	5	0.00%
350	24,150	1,240	1,580	12	0.00%
500	49,500	2,180	2,820	18	0.00%

Schedules held 10 to 100 classes per day (N over five days). The pairwise-check count is five days times n(n−1) ordered pairs per day. Growing the schedule tenfold (50 to 500 classes) raises mean latency from 184 µs to 2,180 µs, about 11.8 times, while the pairwise-check count grows about 110 times. Latency therefore tracks the per-candidate scan of Algorithm 1 rather than the total number of pairs. The 95th percentile stays below 2.9 ms, which is imperceptible in an interactive scheduling screen, and no false positives occurred. Table II lists 37 injected clashes in total (0 + 2 + 5 + 12 + 18). The overlap test is the condition of Lemma 1, implemented as a single database query condition, and unit tests confirm that room, instructor and section clashes are each rejected; missed clashes (false negatives) are not reported separately.

C. Seating allocation throughput

TABLE III
ALGORITHM 2 ALLOCATION TIME AND THROUGHPUT

Facility (grid R × C)	Desks	Cohorts	Time (ms)	Seats/s	Viol. of (4)
Small classroom (5 × 6)	30	2	21	1,428	0
Lecture room (8 × 8)	64	2	42	1,523	0
Examination hall (10 × 12)	120	3	89	1,348	0
Exam complex (5 × 10 × 10)	500	4	382	1,308	0
Campus-wide (25 × 10 × 10)	2,500	8	1,890	1,322	0

Throughput stays between 1,308 and 1,523 seats per second. From 30 to 2,500 seats (83 times more) allocation time rises from 21 ms to 1,890 ms (90 times), consistent with the linear bound of Section IV-D. No violation of invariant (4) was found in the evaluated configurations. The unit tests check this by confirming that consecutive allocated seats alternate between cohorts, which covers horizontal neighbours only; it says nothing about vertical or diagonal neighbours.

D. Verification suite

TABLE IV
INTEGRATION TEST COVERAGE

Domain (target)	Tests	Edge cases exercised	Time (s)	Pass rate
Spatial hierarchy (SpaceManagementService)	32	Aisle gaps, orphan cascading, invalid floors	3.2	100%
Timetable conflicts (TimetableCollisionEngine)	44	Abutting intervals (E1 = S2), subset intervals	4.1	100%
Exam seating (SeatingLayoutService)	38	Odd grids, single-cohort rooms, blocked desks	4.8	100%
Security (JwtService, JwtAuthFilter)	26	Expired tokens, tampered HMAC, RBAC routes	2.7	100%
Total	**140**	Full Spring context slice	**14.8**	**100%**

The suite deliberately exercises the boundaries identified in Section IV: abutting intervals, which Lemma 1 must accept; odd grid dimensions and single-cohort rooms, where the checkerboard leaves the other class empty; and tampered or expired tokens for the security layer.

E. Deployment snapshot

The system was also exercised on a live instance deployed on Railway. Table V lists the records reported by its executive report screen, and Fig. 3 shows the timetable screen with its weekly statistics. The instance holds 11 accounts (8 students and 2 faculty), 25 rooms with 1,180 seats in five blocks, 11 recurring timetable slots (12 hours per week, 11 rooms assigned, none pending) and 3 examination sessions with 16 assigned desks across 3 halls. This is a small pilot dataset, so it shows functional integration of the five modules rather than scale.

TABLE V
DEPLOYED INSTANCE: ACTIVE RECORDS BY MODULE

Module	Primary capability	Active records
1	Authentication, users, JWT RBAC	11 accounts verified
2	Campus, rooms, physical seating	25 rooms, 1,180 seats
3	Timetable and space allocation	11 scheduled slots
4	Exam scheduling, anti-cheating seating	3 exams, 16 desks assigned
5	Student portal and telemetry	Student timetable, exam hall, campus sync

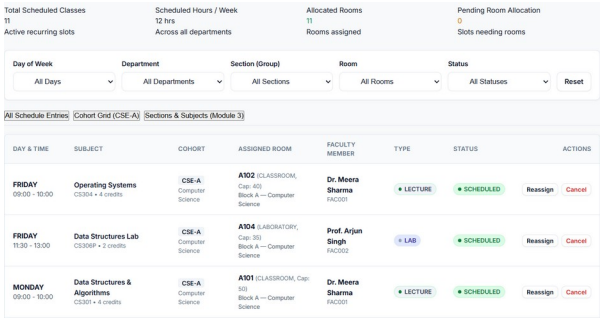


Fig. 3. Timetable and room allocation screen showing weekly statistics and scheduled sessions.

VII. DISCUSSION

A. Deterministic versus stochastic approaches

The metaheuristic and integer-programming methods in Section II optimise soft preferences and can build timetables from scratch, but their running time and results vary with parameters and random seeds [2], [3]. CampusGrid's checks are deterministic and cheap, give the same answer every time and explain themselves through the diagnosis vector. The two styles are complementary: a solver such as UniTime [11] or Timefold [12] could generate a timetable and CampusGrid's engine could validate manual edits to it, while the seat-level model supplies the capacity figures that a solver would otherwise approximate.

B. Limitations of the seating guarantee

Three limitations should be stated plainly. First, invariant (4) excludes only orthogonal neighbours; candidates of the same paper may sit diagonally adjacent, where $D_\infty = 1$. The four-class

variant of Section IV-D removes this at a cost of three quarters of each exam's potential seats. Second, an exam cannot be isolated if it needs more seats than one parity class offers, and splitting exams into two groups that each fit a class is a partition problem that the first-fit-decreasing heuristic solves only approximately, so the algorithm must report infeasibility rather than silently place candidates in the wrong class. Third, adjacency is only one proxy for the opportunity to copy; sight lines, seat rows and invigilator placement are not modelled.

C. Scope and threats to validity

CampusGrid validates sessions and allocates seats; it does not generate timetables or optimise soft constraints, so Table I compares scope, not performance. The instructor vector currently lacks the dedicated index described in Section IV-B. The evaluation uses synthetic workloads on a limited setup, injects clashes rather than using real institutional timetables, and has no baseline such as a naive pairwise scan or a solver, so the latency figures show adequacy for interactive use rather than superiority. Public benchmarks, such as those from the international timetabling competitions, were not used. The live instance shown in Figs. 2 and 3 is a small pilot (8 students, 2 faculty, 11 timetable slots) and was not load-tested. No dedicated micro-benchmark harness such as JMH was used, so absolute timings are indicative. The two-role security model and non-revocable tokens are further limits.

D. Practical implications

For a small or mid-sized institution the system offers an inexpensive route from spreadsheets and door charts to validated schedules and structured seating. It runs as a single container with a standard relational database, so it can be hosted on commodity cloud platforms, and administrators receive concrete explanations of clashes instead of a bare rejection.

VIII. CONCLUSION AND FUTURE WORK

This paper presented CampusGrid, a Spring Boot system that unifies seat-aware space modelling, dual-vector conflict detection and deterministic anti-cheating seating. The seat automaton and effective capacity remove phantom capacity, the half-open interval lemma yields a precise and testable clash definition, and the checkerboard interleaving carries a short correctness proof together with an explicit feasibility condition. In the reported experiments, conflict checks stayed under 3 ms for schedules of 500 classes, seating allocation scaled linearly at about 1,300 seats per second without orthogonal-adjacency violations, and all 140 tests passed.

Future work will (i) add a faculty role and refresh-token rotation or revocation; (ii) implement the four-class variant and a feasibility report in the administrator interface; (iii) add the instructor index and compare against a naive baseline and a solver on public benchmarks; (iv) integrate a solver for timetable generation, with CampusGrid as the validating layer; and (v) extend the platform with IoT turnstiles, QR-coded examination tickets and reinforcement-learning-based scheduling.

ACKNOWLEDGMENT

The authors thank Er. Ram Babu Buri, Assistant Professor, Department of Artificial Intelligence & Data Science, for guidance and supervision throughout this project.

REFERENCES

[1] A. Schaerf, "A survey of automated timetabling," *Artif. Intell. Rev.*, vol. 13, no. 2, pp. 87–127, 1999.

[2] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," *OR Spectrum*, vol. 30, no. 1, pp. 167–190, 2008.

[3] M. C. Chen, S. N. Sze, S. L. Goh, N. R. Sabar, and G. Kendall, "A survey of university course timetabling problem: Perspectives, trends and opportunities," *IEEE Access*, vol. 9, pp. 106515–106529, 2021.

[4] S. N. Jat and S. Yang, "A hybrid genetic algorithm and tabu search approach for post enrolment course timetabling," *J. Scheduling*, vol. 14, no. 6, pp. 617–637, Dec. 2011.

[5] E. Burke, D. Elliman, P. Ford, and R. Weare, "Examination timetabling in British universities: A survey," in *Practice and Theory of Automated Timetabling*, ser. LNCS, vol. 1153. Berlin, Germany: Springer, 1996, pp. 76–90.

[6] R. Qu, E. K. Burke, B. McCollum, L. T. G. Merlot, and S. Y. Lee, "A survey of search methodologies and automated system development for examination timetabling," *J. Scheduling*, vol. 12, no. 1, pp. 55–89, 2009.

[7] E. S. K. Siew, S. N. Sze, S. L. Goh, G. Kendall, N. R. Sabar, and S. Abdullah, "A survey of solution methodologies for exam timetabling problems," *IEEE Access*, vol. 12, pp. 41479–41498, 2024.

[8] S. M. Al-Yakoob, H. D. Sherali, and M. Al-Jazzaf, "A mixed-integer mathematical modeling approach to exam timetabling," *Comput. Manag. Sci.*, doi: 10.1007/s10287-007-0066-8.

[9] E. Köksalmış, C. Garcia, and G. Rabadi, "The optimal exam experience: A timetabling approach to prevent student cheating and fatigue," *Int. J. Oper. Res.*, vol. 21, no. 3, 2014.

[10] C. Beyrouthy, E. K. Burke, J. D. Landa-Silva, B. McCollum, P. McMullan, and A. J. Parkes, "Towards improving the utilization of university teaching space," *J. Oper. Res. Soc.*, 2009.

[11] T. Müller, K. Murray, et al., "University course timetabling and student sectioning system," in *ICAPS 2007 Satellite Demonstrations*. [Online]. Available: https://www.unitime.org

[12] Timefold, "Timefold Solver documentation." [Online]. Available: https://timefold.ai

[13] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[14] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001.

[15] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.

[16] M. Jones, "JSON Web Algorithms (JWA)," IETF, RFC 7518, May 2015.

[17] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, 1999, pp. 81–91.

[18] R. Diestel, *Graph Theory*, 5th ed. Berlin, Germany: Springer, 2017.

[19] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," IETF, RFC 8446, Aug. 2018.